HONEY TABLE

The Algorithm — how the secret hive hides a message

What it is

The Honey Table is a pen-and-paper style cipher for two people who share a secret — classically a **date of birth**. One person hides a short word inside a grid that looks like a jumble of letters and numbers (a honeycomb). The other, knowing the same birthday, lifts the word back out. Anyone else just sees noise.

It takes **two inputs**: the **Honey Key** (the date of birth) and the **Honey Word** (the secret message). The output is the grid — ready to print, photograph, or share as a code.

The shape of the comb

The grid always has **10 rows**, numbered 0–9. The number of columns is **(length of the Honey Word) + 1**: one structure column plus one column for every letter.

Column 0 — the structure column. Read top to bottom it carries the comb's identity:

- **Row 0 (blue)** — the **direction** marker.
- **Rows 1–8 (red)** — the **eight key digits**, the date written as DDMMYYYY.
- **Row 9 (green)** — the **version** number (currently 0).

Direction (1) + eight key digits (8) + version (1) = 10 cells — which is exactly why the comb has 10 rows.

The placement rule

Every letter of the Honey Word gets its **own column** (column 1, 2, 3, …). Each column is labelled with the matching **key digit**, and the letter is dropped into the **row equal to that digit**. Because a digit is always 0–9, the letter always lands somewhere in the 10 rows.

If the word is longer than eight letters, the key digits simply repeat in a cycle.

Worked example

Honey Key = **15-08-1947** (digits **15081947**) • Honey Word = **HAPPY**. Tracing each letter to row = its column's key digit:

Letter	Column	Key digit	Lands at
H	1	1	row 1
A	2	5	row 5
P	3	0	row 0
P	4	8	row 8
Y	5	1	row 1

		1	5	0	8	1
0	0	1	A	P		1
1	1	H				Y
2	5	A	P	P	L	E
3	0	C	A	T		2
4	8	T	I	G	E	R
5	1		A		4	6
6	9	M	A	L	A	Y
7	4	9	W	1	3	
8	7		R		P	4
9	0	H	E	L	L	O

The HAPPY grid for key 15-08-1947. Blue = direction, red = key digits, green = version, orange = the hidden word.

Direction Key digits Version Honey word

Reading it back

To open a comb you only need the birthday. Take the key digits in order. For column 1, jump to the row named by the first digit and read the letter; for column 2, the second digit; and so on across the comb. The letters, in column order, spell the Honey Word.

A **wrong birthday** produces a different list of digits, so the reader jumps to the wrong rows and collects decoy letters — the message stays hidden.

Versions

The version number lives in the green bottom-left cell. Today every comb is **version 0**. The red key-digit cells also act as reserved space, so future versions can change the rules while every comb still announces which version it is — a reader always knows how to open it.

In pseudocode

```
build(word, key, version):
    digits = key as DDMMYYYY        # e.g. 15081947
    cols   = length(word) + 1
    grid   = 10 x cols, filled with random decoys
    grid[row 0][col 0] = direction          # blue
    grid[rows 1..8][col 0] = digits[0..7]   # red
    grid[row 9][col 0] = version            # green
    for i, letter in word:
        d = digits[i mod 8]
        grid[row d][col i+1] = letter        # orange
    return grid

read(grid, key):
    digits = key as DDMMYYYY
    word = ''
    for i in 0 .. (cols-2):
```

```
        d = digits[i mod 8]
        word += grid[row d][col i+1]
    return word
```

Note on secrecy: the Honey Table is an obfuscation puzzle, not strong cryptography — the key digits appear in the comb itself, so its security rests on a partner knowing the method. For a genuinely secret channel, pair it with real encryption.